HacdsDeFi RPC Dokümantasyonu

2026-10-21 Geliştiriciler için

0. Servislere genel bakış

hacds üç HTTP servisi içerir. Bağımsızdırlar ve ayrı ayrı kurulabilirler.

Servis	Varsayılan adres	Protokol	Amaç
Tam düğüm	`43.251.101.101:8082`	HTTP/JSON	Zincir verisi, sandbox salt-okunur çağrıları, işlem doğrulama/yayınlama
Oracle	`43.251.101.101:8123`	HTTP/JSON	Elmas puanlama sertifikalarını imzalar
İndeksleyici	`43.251.101.101:8124`	HTTP/JSON	Borçlu defteri (kim tasfiye edilebilir)

Cüzdan zinciri yerel olarak taramaz. Tüm zincir verisi bu üç servisten gelir.

```
tam düğüm :8082 ──GET──► indeksleyici :8124 ──GET /watchlist──► istemci
      ▲                                                            │
      └──────────────── oracle :8123 (sertifika imzalar) ◄─────────┘
```

Yanıt sözleşmesi: her uç nokta, üst düzeyde `ret` alanı olan JSON döner. `ret: 0` başarı demektir; sıfırdan farklı değer başarısızlıktır (genellikle bir `err` alanıyla birlikte).

Not: başarısızlıklar da **HTTP 200** döner. Gövdedeki `ret` alanına bakmalısınız — durum kodu tek başına söylemez.

---

1. Tam düğüm API'si

1.1 Blok yüksekliği

```
GET /query/block/height/latest
```

```
{"ret":0,"height":792580}
```

1.2 Ortalama ücret

Ücret tahmini için.

```
GET /query/fee/average
```

```
{"ret":0,"purity":"6024000000","purity_unit":232}
```

`purity` = `fee / billing_size (bayt)`. Düğüm kabulü `purity ≥ min_purity` gerektirir (şu anda `6024000000`).

1.3 Bakiye (HAC / elmaslar / varlıklar)

```
GET /query/balance?address={adres}&assets=true&asset_meta=true&diamonds=true
```

Üç anahtar da isteğe bağlıdır:

Parametre	Etkisi
`assets=true`	`assets` dizisini döner (zincir üstü varlıklar)
`asset_meta=true`	Her varlığa `name` / `ticket` / `supply` / `decimal` ekler
`diamonds=true`	`diamonds` döner (elmas başına 6 karakterlik birleşik ad dizisi)

Ölçülen yanıt (adres `15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q`):

```
{
  "ret": 0,
  "list": [{
    "hacash": "217001914996:237",
    "diamond": 7,
    "satoshi": 0,
    "diamonds": "ZUKKXZTTBAAUAYEWKKIXAYYUYTXNNBEUETIVYBYHYX",
    "assets": [{
      "serial": "2000",
      "amount": "4000000000",
```

```
      "decimal": 8,
      "name": "hacds",
      "ticket": "HACDS",
      "supply": "5963298700000000"
    }]
  }]
}
```


Alan	Anlamı
`hacash`	HAC bakiyesi, `değer:birim` biçiminde (`237` = 1:237 hassasiyeti)
`diamond`	Elmas sayısı
`diamonds`	Birleşik elmas adları, **her biri 6 karakter**
`satoshi`	SAT bakiyesi
`assets[].serial`	Varlık serial'i. **hacds =** `2000`
`assets[].amount`	Bakiye. String, JS hassasiyet kaybını önlemek için
`assets[].decimal`	Ondalık basamak. hacds `8` 'dir
`assets[].supply`	Toplam arz

⚠ `amount` **atoms cinsindendir, adet değil.** `4000000000` = 40 hacds. Bkz. bölüm 4.

1.4 Elmas detayı

```
GET /query/diamond?name={6 karakterlik elmas adı}
```


```
{
  "ret": 0,
  "name": "KWTVSH",
  "belong": "VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1",
  "inscriptions": ["hacds"],
  "number": 85662,
  "miner": "1GSL5G6LyuKgg325DxPtfPouaTeTHaZbGh",
  "born": {"height": 522125, "hash": "00000000..."},
  "life_gene": "05e7b8f313761bc8d55ddfe6945b3ba1aedbb4065288bceb5901d24c3cb2a9aa",
  "visual_gene": "aab018d628cb912cc290"
}
```


`belong` mevcut sahibidir. Kontrat adresine eşitse elmas yatırılmıştır.

`visual_gene`, görselin ve puanın girdisidir.

1.5 Sandbox salt-okunur çağrısı (kontrat durumunu okumanın ana yolu)

```
GET /query/contract/sandboxcall?contract={kontrat}&function={fonksiyon}&params={parametreler}&caller={çağıran}
```

Salt-okunur yürütür: durum yazmaz, işlem oluşturmaz, ücret almaz. Ancak kontrat verisi okuyan çağrılar için `caller` **olarak kontrat adresinin kendisini geçin** — aksi halde bazı fonksiyonlar çağıranın bakiyesi olmadığı için başarısız olur (aşağıdaki nota bakın).

Ölçülen yanıt:


```
{"ret":0,"use_gas":377,"gas_use":{"compute":132,"resource":209,"storage":36},"ret_val":120200677111}
```


Alan	Anlamı
`ret`	0 = çağrı başarılı
`ret_val`	Fonksiyonun dönüş değeri (u128, string veya sayı)
`use_gas` / `gas_use`	Bu çağrının tüketimi, tahmin için

⚠ **Not 1:** `caller` **verilmezse başarısız olur.** Verilmeden ölçüldüğünde:


```
{"ret":1,"err":"[revert] address \"1111111111111111111114oLvT2\" balance 0:0 is insufficient, at least 559555:242"}
```


Varsayılan çağıranın bakiyesi sıfırdır ve gaz ön provizyonunu karşılayamaz. `caller={kontrat adresi}` **geçin, çalışır.**

⚠ **Not 2: IP başına eşzamanlılık 2 ile sınırlıdır.** Düğüm eşzamanlı sandbox çağrılarını sınırlar ve sınırı aşan istekler **doğrudan reddedilir** (kuyruklama yoktur):


```
{"ret":1,"err":"sandbox_per_ip_concurrency_exceeded"}
```

İstemciler **sıraya koymalı ve başarısızlıkta tekrar denemelidir**. `Promise.all` ile bir düzine isteği aynı anda göndermeyin.

Salt-okunur kontrat fonksiyonları

Parametreler `değer:tür` biçiminde, birden fazlası virgülle ayrılır.

Fonksiyon	Parametreler	Döner	Not
`amm_reserves`	—	u128	Havuzdaki HAC (**zhu**)
`amm_reserve_ds`	—	u128	Havuzdaki hacds (**atoms**)
`pool_price`	—	u128	Havuz fiyatı = `amm:hac × 1e8 / amm:ds`
`lp_total`	—	u128	Toplam LP payı
`lp_share_of`	`{addr}:address`	u128	Bir adresin LP payı
`lp_pending_hac`	`{addr}:address`	u128	Bekleyen LP ödülü (HAC, zhu)
`lp_pending_ds`	`{addr}:address`	u128	Bekleyen LP ödülü (hacds, atoms)
`total_staked`	`{kind}:u128`	u128	Toplam stake. kind `0`=HAC, `1`=hacds
`treasury_pending`	`{kind}:u128`	u128	Hazine bekleyeni. kind `0`=HAC, `1`=hacds
`balance_hac`	`{addr}:address`	u128	Kontrat içi HAC puanı (zhu)
`balance_ds`	`{addr}:address`	u128	Bekleyen hacds kotası, `due` (atoms)
`loan_status`	`{addr}:address`	u128	Borçlu durumu, aşağıya bakın
`loan_debt_of`	`{addr}:address`	u128	Borç (zhu)
`loan_coll_of`	`{addr}:address`	u128	Teminat (atoms)
`quote_score`	`{name}:bytes,{vg}:bytes,{num}:u128`	u128	Ad + gen + numara için puan

`loan_status` **dönüş anlamları** (önemli — sağlık yüzdesi olarak okumayın):

Değer	Anlamı
`0`	Kredi yok, teminat yok
`999`	Teminat var ama borç yok (güvenli)
diğer	`coll_hac × 100 / debt`, yani **teminat oranı ×100**

Tasfiye koşulu: `debt > 0 && 0 < status < 200` iken tasfiye edilebilir.

Ölçülen örnekler:

```
GET /query/contract/sandboxcall?contract=VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1
    &function=amm_reserves&caller=VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1
→ {"ret":0,"use_gas":377,"gas_use":{"compute":132,"resource":209,"storage":36},"ret_val":120200677111}

GET ...&function=loan_status&params=15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q%3Aaddress&caller={kontrat}
→ {"ret":0,"use_gas":830,"gas_use":{"compute":363,"resource":323,"storage":144},"ret_val":0}
```

> `:` karakteri `%3A` olarak URL kodlanmalıdır.

1.6 Blok özeti

İndeksleyici bunu blokları gezip borçluları bulmak için kullanır.

```
GET /query/block/intro?height={yükseklik}&tx_hash_list=true
```

```
{
  "ret": 0,
  "hash": "0000000000279d55a83105ce01311cd50b06fb5efcb1175e3891a9b46fcf27b1",
  "version": 1,
  "height": 792575,
  "timestamp": 1792537829,
  "nonce": 17,
  "difficulty": "...",
  "miner": "...",
  "reward": "...",
  "tx_hash_list": ["b6488be2b07be8745ac5827e00928ebc840a803d6a132cd9eb573812221e55ce"]
}
```

`tx_hash_list` yalnızca `tx_hash_list=true` verildiğinde döner.

1.7 İşlem sorgusu

```
GET /query/transaction?hash={işlem hash'i}&action=true
```

`action=true` olmadan yalnızca özet döner:

```
{
  "ret": 0,
  "hash": "b6488be2...",
  "hash_with_fee": "cf947b86...",
  "type": 2,
  "timestamp": 1792537576,
  "fee": "245:247",
  "fee_got": "245:246",
  "main_address": "1NsStAq1xt7cuqvEAobcS8T6gqcyS2gnMB",
  "action": 1,
  "block": {"height": 792575, "timestamp": 1792537829},
  "confirm": 5
}
```

`action=true` ile yapılandırılmış bir eylem dizisi eklenir:

```
"actions": [{
  "kind": 4,
  "d": {
    "diamond": "IKEMYN",
    "number": 138592,
    "prev_hash": "0x0000...",
    "nonce": "0x00000001a90d5c45",
    "address": "1EaqdAi9FLCQwLYPLWVtoRM1fZwGezXLTi",
    "custom_message": "0xb0114b5f..."
  }
}]
```

Kontrat çağrıları `kind = 44` **'tür** ve bayt kodları fonksiyon seçicisini içerir:

```
09 14 01 <sig4: 4 bayt> ee
```

`sig4 = sha3_256(fonksiyon adı)[0:4]` . İndeksleyici, bir işlemin hangi kontrat fonksiyonunu çağırdığını böyle belirler.

Not: bu uç noktanın işlemi bulması için onaylanmış olması gerekir (`confirm ≥ 1`); onaylanmamış işlemler görünmez.

1.8 İşlem doğrulama (insan okunur açıklamayla)

```
POST /util/transaction/check?description=true
Content-Type: application/octet-stream
Body: işlemin ham baytları (hex string değil)
```

İmzalı ve imzasız ikisi de çalışır. `description=true` ile sade dille açıklamalar döner.

Ölçülen yanıt:

```
{
  "ret": 0,
  "hash": "37637b89...",
  "hash_with_fee": "14f0c5e3...",
  "type": 3,
  "timestamp": 1792469415,
  "fee": "1:248",
  "fee_got": "1:248",
  "main_address": "15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q",
  "action": 1,
  "gas_max": 99,
  "description": "Main account 15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q pay 1:248 HAC tx fee",
  "actions": [{
    "action": {
      "kind": 16,
      "metadata": {
        "serial": 2000,
        "supply": 5963298700000000,
        "decimal": 8,
        "issuer": "VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1",
        "ticket": "0x4841434453",
        "name": "0x6861636473"
      },
      "protocol_cost": "8:248"
    },
    "description": "Register Asset #2000 \"HACDS\" (name \"hacds\", supply 5963298700000000 atoms, 8 decimals, issuer VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1);
pay 8 HAC protocol fee"
```

```
  }]
}
```

Başarısızlıklar da HTTP 200 döner:

```
{"ret":1,"err":"transaction body invalid"}
```

1.9 İşlem yayınlama

```
POST /submit/transaction
Content-Type: application/octet-stream
Body: imzalanmış işlemin ham baytları
```

Başarı:

```
{"ret": 0, "hash": "işlem hash'i"}
```

Başarısızlık:

```
{"ret": 1, "err": "transaction parse failed"}
```

⚠ `ret:0` **yalnızca havuza girdiğini gösterir, zincire girdiğini değil.** Hacash **300 saniyelik** bloklar hedefler, dolayısıyla ancak bir sonraki blokta yürürlüğe girer.

Onaylamak için `/query/transaction?hash=...` yoklayıp `confirm` alanına bakın veya `/query/balance` ile bakiyeyi izleyin.

1.10 Kontrat depolama ücreti parametreleri

Kontrat dağıtırken veya yeni anahtarların maliyetini tahmin ederken kullanılır.

```
GET /query/contract/storage_fee
```

```
{
  "ret": 0,
  "enabled": true,
  "rule_version": 1,
  "head_height": 792581,
  "target_capacity_blocks": 1000,
  "capacity": 1000000,
  "rate": 1000,
  "remaining": 1000000,
  "p_min": 10,
  "p_max": 10000,
  "periods": 10,
  "k_max": 16384
}
```

`periods` mevcut indirim dönemi sayısıdır. Tam fiyatta değilse, dağıtım eylemini bununla yeniden üretin.

---

2. Oracle API'si

2.1 Sağlık kontrolü

```
GET /health
```

```
{"ok": true, "oracle": "1H3sFZ53HzNVUVN97mjpC9JT8s2PqKMyRu"}
```

`oracle`, oracle hesap adresidir. **Bu adres kontratta sabittir** — başka bir adresle imzalanan sertifikalar kabul edilmez.

2.2 Puanlama sertifikası isteme

Elmas yatırmak veya geri almak için gereklidir.

```
GET /cert?name={6 karakterlik elmas adı}
```

Ölçülen yanıt:

```
{
  "name": "KWTVSH",
  "number": 85662,
```

```
  "visual_gene": "aab018d628cb912cc290",
  "life_gene": "05e7b8f313761bc8d55ddfe6945b3ba1aedbb4065288bceb5901d24c3cb2a9aa",
  "score": 20,
  "hacds_amount": 2000000000,
  "details": {
    "shapes": "Common diamond",
    "styles_or_color": "Double mix",
    "nameliteral": "Common diamond",
    "numberliteral": "Common diamond",
    "score": 20
  },
  "cert_message_hex": "4b5754565348aab018d628cb912cc2900000000000014e9e0000001401",
  "cert_sha3_256": "0fca486f6d7c934006efb7ab926a12928bfda49d09ced3440c27ac2c39a88921",
  "sign_hex":
"023d7424448df611baf27f5188d1e48450082e94bb4d207b1612e22d299bd372d1aff139d57bd79446670c3ef77bab7471a33ada1df5f72adc90e13d316d7bcffc4ce07221ae7730e4b84b51f6320
  "oracle_address": "1H3sFZ53HzNVUVN97mjpC9JT8s2PqKMyRu",
  "version": 1
}
```


Alan	Anlamı
`score`	Elmas puanı. **1 puan = 1 hacds**
`hacds_amount`	Verilen hacds, **atoms** cinsinden (`2000000000` = 20)
`cert_message_hex`	Sertifika mesaj gövdesi (29 bayt), işlemin Message/Blob kısmına girer
`sign_hex`	Oracle'ın o mesaj üzerindeki imzası (97 bayt)
`oracle_address`	İmzalayan hesap

Sertifika uzunluğu = 126 bayt (29 bayt mesaj + 97 bayt imza). Bir işlemde N elmas, `N × 126` bayt sertifika demektir.

Hatalar:

```
{"error": "query name must be 6 chars"}   // HTTP 400
{"error": "..."}                          // HTTP 502, genellikle düğüme ulaşılamıyor
```

⚠ **Elmas adı tam olarak 6 karakter olmalıdır.** Büyük/küçük harf duyarsızdır (sunucu büyük harfe çevirir).

---

3. İndeksleyici API'si

İndeksleyici tam düğümü yalnızca salt-okunur tüketir. **Hiç işlem göndermez, özel anahtar gerektirmez, bakiye gerektirmez.**

Tek bir sorunu çözer: **kontrat borçluları listeleyemez**. Kredi kayıtları adresten oluşturulan anahtarlarda saklanır (`loan:debt:<adres>`); kontratta liste yok, günlük yazılmaz ve düğüm kontrat KV'sini önekle listeleyemez. Dolayısıyla tek yol blokları gezip geçmişte kredi fonksiyonu çağırmış adresleri ortaya çıkarmaktır.

> **Dağıtım durumu**: şu anda `43.251.101.101:8124` `502` döner — servis çalışmıyor.
> Aşağıdaki yanıt biçimleri kaynaktan (`hacds_indexer.py`) alınmıştır; çalıştığında alacağınız biçim budur.

3.1 Sağlık kontrolü

```
GET /health
```

```
{"ok": true, "service": "hacds-indexer", "head": 792580, "watched": 12}
```

3.2 Defter anlık görüntüsü (bir istemcinin ihtiyaç duyduğu tek uç nokta)

```
GET /watchlist
```


```
{
  "ret": 0,
  "head": 792580,
  "node": "http://43.251.101.101:8082",
  "contract": "VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1",
  "watched": 12,
  "withDebt": 3,
  "liquidatable": 1,
  "snapshot": [{
    "address": "1Abc...",
    "health": 150,
    "healthPct": 1.5,
    "debt": 12345,
    "coll": 67890,
    "liquidatable": true,
    "noLoan": false,
```

```
    "first": 792000,
    "last": 792575,
    "calls": 4,
    "fns": {"loan_borrow": 2, "loan_add_collateral": 2},
    "manual": false,
    "note": null
  }]
}
```

Alan	Anlamı
`head`	Taranan yükseklik
`watched`	Defterdeki adres sayısı
`withDebt`	Borcu olanların sayısı
`liquidatable`	Tasfiye edilebilir olanların sayısı
`snapshot[].health`	Ham `loan_status` (`0` / `999` / oran ×100)
`snapshot[].healthPct`	Kat sayısına çevrilmiş (`1.5` = %150). `0` veya `999` iken `null`
`snapshot[].liquidatable`	Sunucuda `debt > 0 && 0 < health < 200` ile değerlendirilmiş
`snapshot[].first/last`	İlk / en son kredi çağrısında göründüğü blok yüksekliği
`snapshot[].fns`	Fonksiyon başına çağrı sayısı
`snapshot[].manual`	Elle mi eklendi

İstemcilerin tasfiye edilebilirliği kendileri hesaplaması gerekmez — doğrudan `liquidatable` kullanın.

3.3 Yalnızca anlık görüntü

```
GET /snapshot
```

```
{"ret": 0, "list": [ /* yukarıdaki snapshot dizisinin aynısı */ ]}
```

3.4 Şimdi eşitle

```
GET /sync
```

Bir tarama tetikler ve sonucu döner.

3.5 Defteri elle yönetme

Taranacak geçmişin olmadığı ilk günlerde veya izlemek istediğiniz adresleri eklemek için.

```
GET /watch/add?address={adres}&note={not}
GET /watch/remove?address={adres}
```

`/watch/add` güncellenmiş adres listesini döner:

```
{"ret": 0, "list": ["1Abc...", "1Def..."]}
```

`/watch/remove`:

```
{"ret": 0, "removed": true}
```

`address` eksikse HTTP 400 döner:

```
{"ret": 1, "err": "address required"}
```

⚠ **Defter tek kaynak olmalıdır.** Birden fazla indeksleyici örneği çalıştırıyorsanız, `/watch/add` ve `/watch/remove` aynı örneğe gitmelidir, yoksa defterler ayrışır.

3.6 Dağıtım

```
python3 hacds_indexer.py --port 8124 --node http://<node>:8082
```

- Sıfır üçüncü taraf bağımlılığı, Python 3 standart kütüphanesi
- Eşzamanlı tarama için `--workers` (varsayılan 4; ölçülen 2,6 → 6,6 blok/sn)
- Artımlı: `lastScanned` kaydeder, sonra yalnızca yeni blokları tarar
- Defter atomik yazılır (`tmp` + `os.replace`)

- systemd birimi: `hacds_exchange/indexer/hacds_indexer.service`

---

4. Borsa listeleme entegrasyonu (özel bölüm)

Bu bölüm hacds'i entegre edecek borsalar, cüzdanlar ve piyasa verisi servisleri içindir.

4.0 Önce bir şeyi unutmayın: amount atoms cinsindendir

hacds, `decimal = 8` bir varlıktır. Zincirdeki ve arayüzlerdeki `amount`, **en küçük birim olan atoms**'tır, adet değildir.

```
1 hacds = 100000000 atoms (1e8)
toplam arz = 5963298700000000 atoms = 59.632.987 hacds
```

Bunu yanlış yapmak yüz milyon kat sapma demektir.

4.1 Varlık kimliği (listeleme kaydı için)

Öğe	Değer
serial	**2000**
ticket	`HACDS`
name	`hacds`
decimal	**8**
toplam arz	`5963298700000000` atoms
issuer	`VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1`
ihraççı adresi	`15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q`

⚠ Zincirde ticket'ı da `HACDS` olan başka bir varlık vardır (serial `1028`, adı "HACD Fragments", bizimle ilgisiz).

Protokol, ticket veya ad benzersizliğini kontrol etmez — yalnızca serial'in alınıp alınmadığına bakar.

`serial = 2000` ile eşleştirin. Ticket ile eşleştirmeyin.

4.2 Yatırma tespiti

Yalnızca serial ile eşleştirmeyin — serial=2000 *ve* issuer kontrol edin.

Blokları yoklayın veya abone olun, sonra her işlemdeki varlık eylemlerini ayrıştırın:

```
GET /query/block/intro?height={yükseklik}&tx_hash_list=true
  → tx_hash_list
GET /query/transaction?hash={hash}&action=true
  → actions[], varlık transferlerini süzün
```

Varlık transferi eylemi `kind`:

kind	Eylem	Anlamı
17	`TransferAssetTo`	Kontrata / basit hedefe transfer
19	`TransferAssetFromTo`	A'dan B'ye transfer

Eylemin `d` alanından `serial` / `amount` ve gönderen/alıcı adreslerini okuyun. **`serial == 2000` hacds'tir** ve ayrıca `issuer`'ın kontrat adresi olduğunu kontrol etmeniz önerilir.

Yatırılacak miktar:

```
kullanıcının gördüğü hacds adedi = amount / 1e8
```

Örneğin `amount = 4000000000` alındığında **40 hacds** yatırılır.

⚠ Büyük tam sayı kullanın. `5963298700000000`, JS `Number.MAX_SAFE_INTEGER` (`9007199254740991`) değerinin %66 üzerindedir. JSON'daki `amount` bir string'tir — **önce double'a çevirmeyin**. JS'te `BigInt`, Go'da `big.Int`, Java'da `BigInteger`, Kotlin'de `BigInteger` kullanın (`Long` yetmez).

4.3 Çekim oluşturma

hacds standart bir değiştirilebilir varlıktır, dolayısıyla çekim sıradan bir varlık transferidir (`kind = 19`, `TransferAssetFromTo`).

Parametreler:

Parametre	Değer
`serial`	`2000`

Parametre	Değer
`amount`	çekilecek atoms (= adet × 1e8)
`from`	borsa sıcak cüzdan adresi
`to`	kullanıcı adresi

Not: `from`, işlemi başlatan hesap olmalı ya da yetkilendirilmiş olmalıdır; borsanın sıcak cüzdan anahtarını yönetip yerelde imzalaması gerekir.

Oluşturmanın iki yolu:

1. Type-3 işlemi **Hacash'in resmî araçlarıyla** kendiniz kurun
2. **Bu depodaki Rust motoruna** bakın: `hacds_txbuild` (`transfer asset-from-to` alt modu); kaynak Type-3 tel formatını, ücret yinelemesini ve imzalamayı zaten yönetir

> Uzaktan imzalama uç noktası sunmuyoruz. **Düğümün `/util/transaction/sign` uç noktası özel anahtarınızın HTTP üzerinden gönderilmesini gerektirir** — hiçbir koşulda kullanmayın.

4.4 Yayınlama ve onay

```
POST /submit/transaction        # ikili gövde, hex değil
```

```
{"ret": 0, "hash": "..."}
```

Onay akışı:

1. `ret:0` → havuza girdi, **henüz onaylanmadı**
2. `GET /query/transaction?hash={hash}` yoklayın
3. `confirm` alanına bakın; `≥ 1` zincire girdi demektir
4. Veya doğrudan `GET /query/balance` ile bakiye değişimini kontrol edin

Blok hedefi 300 saniyedir. Yatırma/çekim risk onaylarınızı bu ritme göre tasarlayın. Bugün zincirde ölçülen: yaklaşık 5 dakikada bir blok.

4.5 Ücretler ve gaz

- **Ücret**: işlem bayt boyutuna göre alınır. `GET /query/fee/average` ile `purity` alın; `fee = purity × billing_size`.

Ücreti artırmak bayt boyutunu biraz büyütür (ücret kodlaması uzar), bu yüzden **yineleyin**: oluştur → gerçek bayt boyutunu oku → eksikse yeniden oluştur. Bu depodaki `fee.js` bu yinelemedir; genellikle 1–3 turda yakınsar.

- **gaz**: `gas_max` baytı, varsayılan `99` yani protokol sınırı `111.911`. Borsa transfer işlemleri kontrata dokunmaz, dolayısıyla gaz tüketimi çok düşüktür.

4.6 Bilmeniz gereken üç katı sınır

① VM gazı, `gas_max` ile ilgisiz üç bağımsız kovaya ayrılır

Kova	Sınır
compute	18.009
resource	6.100
storage	111.911

Aşıldığında `gas_max`**'ı artırmak hiçbir işe yaramaz**. Hata şöyle görünür:

```
OutOfGas(31): resource gas limit exceeded: used 6101 > limit 6100
```

② Kontrat yatırma işlemi başına en fazla bir HACD

Her elmasın sertifikasını doğrulamak zincir üstü puan yeniden hesaplaması gerektirir, yaklaşık 3.000 resource; iki elmas 6.101 > 6.100 eder.

Borsa "elmas karşılığı hacds" hizmeti veriyorsa **işlem başına bir elmas olacak şekilde bölmek zorundadır**. Atomik işlemle birleştirmek de işe yaramaz (tek gaz bütçesini paylaşırlar).

③ Sandbox çağrıları IP başına 2 eşzamanlılıkla sınırlıdır

Kontrat durumu okumalarını paralelleştirmeyin; sınırı aşan istekler doğrudan reddedilir (kuyruk yok):

```
{"ret":1,"err":"sandbox_per_ip_concurrency_exceeded"}
```

4.7 Piyasa yapıcılığı / kotasyon için gereken veriler

İhtiyacınız	Uç nokta
Havuzdaki HAC	`sandboxcall` → `amm_reserves` (zhu; ÷1e8 = HAC)

İhtiyacınız	Uç nokta
Havuzdaki hacds	`sandboxcall` → `amm_reserve_ds` (atoms)
Spot fiyat	`sandboxcall` → `pool_price` = `amm:hac × 1e8 / amm:ds`, yani **hacds başına HAC**
Toplam arz	`/query/balance?address={kontrat}&assets=true` içinde serial 2000'in `supply` değeri
Tasfiye edilebilir pozisyonlar	indeksleyici `GET /watchlist`

Kotasyon dönüşümü:

```
hacds başına HAC cinsinden havuz fiyatı = pool_price / 1e8
örn. pool_price = 301355749 → 3,01355749 HAC/hacds
```

4.8 Mutabakat

Her an kontrol edebileceğiniz zincir üstü özdeşlik:

```
kullanıcı varlıkları + kontrat varlıkları = toplam arz
```

Üç sayı da alınabilir:

```
# toplam arz
GET /query/balance?address={kontrat}&assets=true     → supply
# kontrat varlıkları
GET /query/balance?address={kontrat}&assets=true     → amount
# kullanıcı varlıkları
GET /query/balance?address={kullanıcı}&assets=true   → amount
```

Ölçülen örnek (2026-10-21):

Öğe	atoms
toplam arz	`5963298700000000`
kontrat varlıkları	`5963294700000000`
kullanıcı varlıkları	`4000000000`
kontrat + kullanıcı	`5963298700000000` ✓

4.9 Listeleme entegrasyon kontrol listesi

- [ ] Varlığı **serial 2000** ile eşleştirin, ayrıca issuer'ı doğrulayın
- [ ] `amount`'ı baştan sona büyük tam sayı olarak tutun; asla double'dan geçirmeyin
- [ ] Kullanıcıya adet gösterirken `1e8`'e bölün
- [ ] Yatırmaları `kind 17/19` varlık eylemlerini ayrıştırarak tespit edin
- [ ] Çekimleri `TransferAssetFromTo(2000, amount, hotWallet, user)` olarak kurun
- [ ] `POST /submit/transaction`'a ikili gövdeyle yayınlayın
- [ ] `confirm ≥ 1` ile onaylayın, 5 dakikalık blok gecikmesini hesaba katın
- [ ] Ücretleri yinelemeli tahmin edin (`purity × size`)
- [ ] Sandbox okumalarını sıralayın (IP başına eşzamanlılık ≤ 2)
- [ ] Elmas işin içindeyse **işlem başına bir tane**
- [ ] "kullanıcı + kontrat = toplam arz" özdeşliğiyle mutabakat yapın

---

5. Hata yönetimi

5.1 Kontrat dönüş kodları

Bir sandbox çağrısı sıfırdan farklı `ret_val` döndüğünde bu, kontrat fonksiyonunun içindeki `return N`'dir. Sık kullanılan eşlemeler:

Fonksiyon	Kod	Anlamı
genel	1	Kontrat duraklatıldı (`cfg:paused = 1`)
`swap_hac_to_ds`	11	HAC puanı yetersiz
`add_liquidity`	2	Bir taraf 0
`add_liquidity`	23	Havuz boş değil ama pay sıfır (ölçek koruması)
`add_liquidity`	24	Hesaplanan pay 0 (oran çok küçük)
`add_liquidity`	26	İlk yatırım: bir taraf `1e18` sınırını aşıyor

Fonksiyon	Kod	Anlamı
`remove_liquidity`	25	LP payı yetersiz
`loan_borrow`	40	Borç limiti aşıldı (`(debt+hac)×2 > coll_hac`)
`loan_repay`	41	Borç 0
`loan_liquidate`	42	Adresin borcu yok
`loan_liquidate`	43	Tasfiye çizgisi aşılmamış (tasfiye edilemez)
`Set*` (yönetici)	1	Yönetici değil
`Set*` (yönetici)	2	Parametre aralık dışı
`PermitAsset`	11	Bekleyen hacds yetersiz
`PermitHACD`	12	Geri alım için hacds yetersiz ve otomatik tamamlama başarısız

Tam tablo `wallet-app/contract-errors.js` içindedir.

5.2 Düğüm / sandbox hataları

String ile eşleştirilir. Sık görülen kategoriler:

Ham mesaj	Anlamı	Ele alınışı
`invalid address argument` / `InvalidChecksum`	Adres biçimi veya sağlama basamağı hatalı	Adresi tamamen yeniden alın
`balance ... is insufficient`	Çağıran gaz ön provizyonunu karşılayamıyor	Sandbox çağrılarında `caller={kontrat adresi}` geçin
`sandbox_per_ip_concurrency_exceeded`	Eşzamanlılık sınırı aşıldı	Sıralayın ve tekrar deneyin
`resource gas limit exceeded`	resource kovası aşıldı	İşlemi bölün; **gas_max'ı artırmak işe yaramaz**
`compute gas limit exceeded`	compute kovası aşıldı	İşlemi bölün
`storage gas limit exceeded`	storage kovası aşıldı	Daha az yeni anahtar veya daha kısa kiralama süresi
`OutOfGas`	Gaz bitti	gas_max'ı artırın
`CallNotExist`	Kontratta böyle bir fonksiyon yok	Fonksiyon adını kontrol edin
`NotFindContract`	Kontrat bulunamadı	Kontrat adresini kontrol edin
`state changed`	Bu arada zincir durumu değişti	Tekrar deneyin
`transaction parse failed`	İşlem gövdesi ayrıştırılamadı	hex / baytlar eksik
`ECONNREFUSED` / `timeout`	Servise ulaşılamıyor	Servisin çalışıp çalışmadığını kontrol edin

Kilit nokta: bunların çoğu **HTTP 200 +** `{"ret":1,"err":"..."}` olarak döner. Yalnızca HTTP durumuna bakmak bunları kaçırır.

---

6. Hızlı başvuru

Uç noktalar

```
# tam düğüm :8082
GET  /query/block/height/latest
GET  /query/fee/average
GET  /query/balance?address=&assets=true&asset_meta=true&diamonds=true
GET  /query/diamond?name=
GET  /query/contract/sandboxcall?contract=&function=&params=&caller=
GET  /query/block/intro?height=&tx_hash_list=true
GET  /query/transaction?hash=&action=true
GET  /query/contract/storage_fee
POST /util/transaction/check?description=true     # ikili gövde
POST /submit/transaction                          # ikili gövde

# oracle :8123
GET  /health
GET  /cert?name=

# indeksleyici :8124
GET  /health
GET  /watchlist        # bir istemcinin ihtiyaç duyduğu tek şey
GET  /snapshot
GET  /sync
GET  /watch/add?address=&note=
GET  /watch/remove?address=
```

Sabitler

```
hacds serial         2000
hacds decimal        8
hacds toplam arz     5963298700000000 atoms
1 hacds              100000000 atoms
kontrat              VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1
hazine               1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK
oracle hesabı        1H3sFZ53HzNVUVN97mjpC9JT8s2PqKMyRu
SCORE_SCALE          1e8 (1 puan = 1 hacds)
COLL_RATIO           2 (%50 teminat oranı, tasfiye çizgisi 200)
AMM komisyonu        %0,5 (LP %50 / stake %30 / hazine %20)
kredi oranı / tasfiye primi  yıllık %5 / %2
sertifika uzunluğu   126 bayt (mesaj 29 + imza 97)
VM kova sınırları    compute 18.009 / resource 6.100 / storage 111.911
blok hedefi          300 saniye
sandbox eşzamanlılık IP başına 2
```

Birim dönüşümü

```
atoms → hacds adedi : amount / 1e8
hacds adedi → atoms : adet × 1e8
zhu   → HAC         : zhu / 1e8
havuz fiyatı (HAC/hacds) : pool_price / 1e8
```